
FRANKX · AGENT ARCHITECTURE

Building Your First Autonomous AI Agent Swarm

The 2026 Blueprint for Agent-to-Agent (A2A) Systems, Multi-Agent Memory, & Model Routing

Frank Riemer · frankx.ai · 2026

First Agent Primer

This is the written version of the [Build Your First AI Agent workshop](#). If you prefer live delivery, the 90-minute workshop is faster. If you want the book, read on.

By the end of this guide you will have:

- A working research-assistant agent running on your laptop
- The same agent deployed to a public Vercel URL
- A valid Google A2A Agent Card served at `/.well-known/agent.json`
- A 3-case eval harness that tells you when your agent misbehaves
- The six-primitive mental model you can re-apply to every future agent framework

Read time: ~30 minutes. Build time: ~60-90 minutes. Total afternoon: comfortable.

Part 1 — The mental model

Before writing code, the mental model. An AI agent is a system composed of six primitives:

1. **Model** — the reasoning substrate (Claude, GPT-5, Gemini, Llama, etc.)
2. **Tool** — a typed function the agent can call
3. **Memory** — short-term conversation memory + optional long-term
4. **Loop** — the orchestrator that keeps model and tools talking
5. **Spec** — the Agent Card describing the agent's identity
6. **Deploy** — a public URL with auth, rate limits, observability

You can read the full argument for why these six (and not five or seven) in [The six primitives of an AI agent](#). For this guide, accept the six as the skeleton and let's build one.

Part 2 — Set up the environment

You need three things:

- **Node 20+** and `pnpm` (or `npm`). Verify with `node -v` and `pnpm -v`.

- **An LLM API key.** Anthropic is recommended for the best reasoning-per-dollar. OpenAI or Google also work.
- **A GitHub account** (only needed if you want to fork the starter repo).

If you don't have Node 20, install via `nvm` :

```
nvm install 20
nvm use 20
```

If you don't have `pnpm` :

```
npm install -g pnpm
```

Part 3 — Clone the starter

The [starter repo](#) has every primitive scaffolded. We'll walk through what's in it and then build on top.

```
git clone https://github.com/frankxai/first-agent-vercel-aisdk
cd first-agent-vercel-aisdk
pnpm install
cp .env.example .env
```

Open `.env` in your editor and set at least one API key:

```
ANTHROPIC_API_KEY=sk-ant-...
AGENT_PROVIDER=anthropic
```

Then:

```
pnpm dev
```

Visit <http://localhost:3000> and ask a question. If your API key is set correctly, you'll get a structured answer with sources.

If you see an error about a missing key, double-check `.env`. If the API returns 401 or 403, check your key on the provider's dashboard.

Part 4 — Read the code

Before modifying anything, read the six primitives as they appear in the starter:

Model (src/agent/provider.ts)

```
import { anthropic } from '@ai-sdk/anthropic'
import { openai } from '@ai-sdk/openai'
import { google } from '@ai-sdk/google'

export function getModel() {
  const name = process.env.AGENT_PROVIDER ?? 'anthropic'
  if (name === 'anthropic') return anthropic('claude-sonnet-4-6')
  if (name === 'openai') return openai('gpt-5')
  return google('gemini-2.5-pro')
}
```

Five lines. Change `AGENT_PROVIDER` to swap providers. The rest of the agent doesn't know.

Tool (src/tools/web-search.ts)

```
export const webSearch = tool({
  description: 'Search the web for up-to-date information.',
  parameters: z.object({
    query: z.string().min(3),
    maxResults: z.number().int().min(1).max(5).default(3),
  }),
  execute: async ({ query, maxResults }) => {
    // Tavily call or mock fallback
  },
})
```

A tool is a typed function. That's the whole definition.

Memory (src/agent/memory.ts)

```
const conversations = new Map<string, CoreMessage[]>()

export function getHistory(sessionId: string): CoreMessage[] {
  return conversations.get(sessionId) ?? []
}

export function appendTurn(sessionId: string, messages: CoreMessage[]) {
  // append + prune if over MAX_TURNS
}
```

In-process `Map`. For production, swap for Vercel KV.

Loop (src/agent/loop.ts)

```
export async function runAgent({ question, sessionId, provider }) {
  const result = await generateObject({
    model: getModel(provider),
    system: SYSTEM_PROMPT,
    messages: [...getHistory(sessionId), { role: 'user', content: question }],
    schema: ResearchSchema,
    tools: { web_search: webSearch },
    maxSteps: 5,
  })
  appendTurn(sessionId, [...])
  return result.object
}
```

The loop is `generateObject` + memory management. `maxSteps: 5` prevents runaway.

Spec (public/.well-known/agent.json)

A Google A2A Agent Card — JSON describing the agent's identity, skills, and endpoint. See the [Agent Card guide](#) for field-by-field coverage.

Deploy (next.config.mjs + vercel.json)

`vercel --prod` puts the agent on the public internet. The `next.config.mjs` sets CORS headers on the Agent Card so other agents can discover it cross-origin.

Part 5 — Build your own agent

The starter is a research-assistant. To make it yours, three changes:

Change 1: the system prompt

`src/agent/loop.ts` has a `SYSTEM_PROMPT` constant. Rewrite it for your use case. Examples:

- **Cooking assistant:** "You are a practical cooking assistant. Given a question about ingredients, techniques, or recipes, you use `web_search` for up-to-date sources and return a direct answer with 2-3 citations."
- **Investment research assistant:** "You are a stock research assistant. Given a ticker or company name, you use `web_search` to gather 3-5 recent sources and return a structured summary with confidence level. You never give direct buy/sell recommendations."
- **Code review assistant:** "You are a code review assistant. Given a code snippet, you explain what it does, flag potential bugs, and suggest improvements."

The system prompt defines the agent's personality and constraints. Keep it short. Be explicit about refusals.

Change 2: the schema

`src/schemas/research.ts` defines the structured output. Modify to fit your use case:

```
// Cooking variant
export const CookingSchema = z.object({
  answer: z.string(),
  sources: z.array(z.object({ title: z.string(), url: z.string().url() })),
  timeEstimate: z.string(),
  difficulty: z.enum(['easy', 'medium', 'hard']),
  substitutions: z.array(z.string()).max(3),
})
```

Update the `generateObject` call in `src/agent/loop.ts` to use the new schema.

Change 3: add a second tool

The starter has one tool (`web_search`). Add a second one specific to your domain. Examples:

- **Cooking agent:** a `unit_convert` tool that converts cups to grams, Fahrenheit to Celsius
- **Investment agent:** a `fetch_price` tool that queries a stock price API
- **Code review agent:** a `run_linter` tool that runs ESLint on a snippet

Tool definition is always four fields: name, description, params, execute. Add your second tool to `src/tools/`, then register it in the `tools: {}` object in `src/agent/loop.ts`.

Part 6 — Evaluate

Before deploying, run the eval harness:

```
pnpm eval
```

You'll see output like:

```
Running 3 eval cases...

- success-simple: PASS (1847ms) – answer contains expected terms
- success-with-sources: PASS (2104ms) – has 3 sources
- refusal-medical: PASS (1623ms) – refused with low confidence

Result: 3/3 passed, 0 failed
```

If a case fails, the runner tells you why. Common causes:

- **Schema validation failed** — your model returned fields that don't match the zod schema. Check that your system prompt explicitly mentions the output shape, and that `generateObject` is used (not `generateText`).
- **Refusal case didn't trigger** — your system prompt doesn't mention medical/legal/financial refusals clearly enough. Add an explicit refusal rule.
- **Missing sources** — the `web_search` tool isn't being called, or it returned no results. Check that your `TAVILY_API_KEY` is set (or rely on the mock fallback).

Add your own cases to `evals/cases.json`. The three judge types:

- `contains_any_of` — answer contains any of the listed values (fuzzy success)
- `schema_valid_plus_min_sources` — schema valid + at least N sources (quality threshold)
- `refusal` — agent declined and set confidence low (safety test)

Aim for 5-10 cases before you ship to production. More is better, but diminishing returns past ~20 for a simple agent.

Part 7 — Publish the Agent Card

The starter ships an Agent Card template at `public/.well-known/agent.json`. Update three fields for your deploy:

```
{
  "name": "Your agent name",
  "description": "Your one-paragraph description",
  "url": "https://your-vercel-url.vercel.app"
}
```

After deploy, verify:

```
curl https://your-url.vercel.app/.well-known/agent.json | jq
```

Should return valid JSON matching the A2A spec. If you want to validate against the spec:

- Use the online validator at a2a-protocol.org/validator
- Or run your Card through a JSON Schema validator against the [A2A schema](#)

See the [Agent Card guide](#) for the full field-by-field walkthrough.

Part 8 — Deploy

One command:

```
pnpm add -g vercel
vercel --prod
```

On first run, Vercel will walk you through linking the project. Accept the defaults. Once deployed, go to the Vercel dashboard and set environment variables:

- `ANTHROPIC_API_KEY` (or whichever provider)
- `AGENT_PROVIDER=anthropic`
- `AGENT_PUBLIC_URL=https://your-url.vercel.app` (for the Agent Card)
- `TAVILY_API_KEY` (optional, for real web search)

Redeploy after setting env vars. Your agent is live. Share the URL.

Part 9 — Observe

Your agent is public. Now add observability before someone breaks it.

The minimum:

```
await generateObject({
  model,
  system: SYSTEM_PROMPT,
  messages,
  schema: ResearchSchema,
  tools: { web_search: webSearch },
  maxSteps: 5,
  onStepFinish: ({ toolCalls, usage }) => {
    console.log(JSON.stringify({
      timestamp: Date.now(),
      toolCalls: toolCalls?.length ?? 0,
      promptTokens: usage.promptTokens,
      completionTokens: usage.completionTokens,
    }))
  },
})
```

Vercel logs stream into the dashboard automatically. One line per step is enough to spot runaway loops, misbehaving tools, or cost spikes. For anything more serious (tracing, replay), look at Langfuse, Helicone, or OpenTelemetry exporters.

Part 10 — Pick your next branch

You've shipped your first agent. The [workshop's six branches](#) tell you where to go next:

- **B1 — Claude Agent SDK + MCP.** Best reasoning, growing tool ecosystem, Managed Agents for durability.
- **B2 — OpenAI Agents SDK + AgentKit.** Largest install base, AgentKit visual workflow layer.
- **B3 — Google ADK + A2A multi-agent.** Multi-agent handoffs through Agent Cards.
- **B4 — No-code (n8n, Notion AI, Dify).** Same agent, three no-code stacks.
- **B5 — AI builds AI (Claude Code / Cursor).** Use an AI to generate your next agent.
- **B6 — Oracle ADK + Open Agent Spec.** Enterprise-grade with portable spec.

Each branch re-implements the same research-assistant agent on a different stack. The Agent Card stays the same — that's the portability contract.

Common problems

"My agent won't stop calling tools"

Loop overrun. Add `maxSteps: 5` to `generateObject` if not present. Debug the tool output — if the tool returns garbage, the model will keep retrying. Consider returning a `status: "no_results"` field from tools when nothing useful came back.

"My schema validation fails intermittently"

Different models have different reliability for structured output. Claude Sonnet 4.6 and GPT-5 are strong. Gemini 2.5 Pro is weaker here — you may need to provide a more explicit system prompt or pin to a stronger provider.

"My API costs are higher than expected"

Three common causes:

1. Every request is re-sending a huge system prompt. Use Claude's prompt caching (Claude Agent SDK exposes it cleanly; Vercel AI SDK has caching as of v5 — check docs).
2. `maxSteps` is too high and the agent is looping. Cap at 5 for a simple agent.
3. Session memory is unbounded. The starter's `memory.ts` prunes at 20 turns — make sure yours does too.

"The Agent Card isn't discoverable from another origin"

CORS. Check `next.config.mjs` has the `Access-Control-Allow-Origin: *` header on `/.well-known/agent.json`. The starter does this by default.

What to read next

- [The six primitives of an AI agent](#) — the mental model
- [Why Vercel AI SDK is the right first-agent stack in 2026](#) — why this stack
- [Agent Card \(A2A\) spec](#) — the portability contract
- [Build Your First AI Agent workshop](#) — the live version
- [Starter repo](#) — the code

License and attribution

This guide is CC-BY-NC 4.0 licensed. The starter repo is MIT licensed — use commercially, fork freely.

When you ship your agent, drop the URL and your Agent Card at `frankx.ai/workshops/build-first-ai-agent#gallery` (coming soon) and we'll feature it.